C++ Systems Engineering Roadmap

Operating Systems (Primary Focus) & Backend Engineering

Prepared for Maimo Harris Shaalanyuy

Senior VAPT Engineer, Cyenetic Solutions LTD | Skye8 | HR-Tech

August 2026

Table of Contents

Overview & Career Vision

You already bring offensive security, red team, and backend development experience (Django/Express), plus solid Linux, networking, and API knowledge. The missing piece is low-level systems programming — exactly where C++ shines.

Becoming proficient in C++ opens doors to operating systems, hypervisors, security tooling, malware analysis, EDR/XDR products, game engines, databases, high-performance backend systems, browser engines, and compilers.

This roadmap is built around three specialization legs that are really the same systems-programming discipline applied at different scales: Operating Systems (kernel-level theory and implementation), Embedded Systems (the same concepts applied bare-metal, under tight resource constraints — directly building on your existing Arduino/ESP32 work), and Backend Engineering (the same concepts again, but at the opposite end of the spectrum with abundant resources and distributed scale). Your existing security expertise runs through all three as a long-term differentiator, not a bonus tacked on at the end.

Target: Systems Security Engineer — someone who builds operating systems, embedded/firmware platforms, backend infrastructure, and security software, not just assesses them.

Starting Point (Concurrent Foundations)

Rather than a strict sequence, begin these four tracks in parallel — they reinforce each other:

- C++ fundamentals: syntax → pointers/references → memory (stack/heap) → classes/OOP → STL containers
- Data Structures & Algorithms: implemented from scratch in C++, then compared against STL
- OS Concepts (theory): processes/threads, virtual memory, scheduling, file systems, synchronization
- Application/Software Design: SOLID principles, core design patterns, basic UML

Suggested weekly rhythm: 3–4 days on C++ + DS&A together (implement a structure, solve 2–3 problems using it), 1–2 days on OS theory + notes, and one small design-pattern example applied to your own code each week.

Phase-by-Phase Roadmap

Phase 1 — Modern C++ Fluency

Duration	6–8 weeks
Core Topics	Variables, references, pointers, memory, arrays, structs, classes, OOP, templates, RAII, smart pointers, STL, iterators, lambdas, move semantics, exceptions (target C++17/20)
Projects	Calculator, Shell, CLI File Explorer, HTTP Parser, JSON Parser
Resources	C++ Primer; Effective Modern C++; learncpp.com

Phase 2 — Computer Science Fundamentals (DS&A)

Duration	Ongoing, alongside Phase 1
Core Topics	Arrays/strings, linked lists, stacks/queues, hash tables, trees (BST → balanced), graphs, heaps, dynamic programming, complexity analysis
Projects	Implement each structure from scratch in C++ before relying on STL's version
Resources	CLRS (theory); NeetCode & LeetCode (practice — aim for depth over volume, ~3-5 problems per structure)

Phase 3 — Computer Architecture & Linux Internals

Duration	2–3 months
Core Topics	x86-64 assembly, registers, stack/heap, calling conventions, cache, branch prediction, SIMD, pipelines; ELF, processes, threads, scheduler, signals, pipes, shared memory, mmap, fork/exec, ptrace
Projects	Mini emulator, simple assembler; rebuild ps, top, kill, grep, cat, cp, mv in C++
Resources	Computer Systems: A Programmer's Perspective (CS:APP)

Phase 3.5 — Embedded Systems Fundamentals (New Specialization Leg)

Duration	2–3 months, alongside Phase 4
Core Topics	Bare-metal C++ (no STL/heap allocation in hot paths, volatile, memory-mapped I/O); ARM Cortex-M (STM32) architecture: registers, clock trees, GPIO, interrupts/NVIC, DMA; UART, SPI, I2C, CAN bus; RTOS concepts via FreeRTOS (tasks, queues, semaphores, priority inversion — the bridge back to OS scheduling theory); toolchain: arm-none-eabi-gcc, OpenOCD, JTAG/SWD debugging, linker scripts
Projects	Bare-metal LED blink -> button interrupt -> UART driver (no framework abstractions); hand-rolled minimal preemptive scheduler for Cortex-M (bridges embedded and OS work); port your existing smart-home project's core logic from Arduino to bare-metal STM32; CAN bus sniffer/logger
Resources	STM32 reference manuals; FreeRTOS documentation; "Making Embedded Systems" (Elecia White)

Phase 4 — Operating Systems Deep Dive

Duration	3–4 months (centerpiece phase)
Core Topics	Virtual memory, paging, segmentation, TLB; CPU scheduling & context switching; mutexes, semaphores, spinlocks; file systems, journaling, block devices; sockets/TCP-IP; interrupts, kernel, syscalls, boot process
Projects	Study and build along with xv6 (MIT teaching OS) or a from-scratch kernel via OSDev.org
Resources	Operating Systems: Three Easy Pieces (OSTEP, free online)

Phase 5 — Application & Software Design

Duration	Parallel, ongoing
Core Topics	SOLID principles, common design patterns (Factory, Observer, Strategy, Singleton), basic UML for communicating designs
Projects	Apply one pattern per week to an existing project
Resources	Head First Design Patterns

Phase 6 — Backend Engineering

Duration	2–3 months, can run parallel to Phase 4
Core Topics	HTTP/HTTP2/HTTP3, TCP/UDP, TLS, REST, gRPC, WebSockets, OAuth/JWT, Redis, PostgreSQL/MySQL, thread pools, async I/O, coroutines
Projects	REST API, chat server, distributed cache, load balancer, multithreaded HTTP server (compare hand-rolled sockets vs Boost.Asio, or try Drogon/Seastar)
Resources	Boost.Asio docs; Drogon; Seastar

Phase 7 — High-Performance C++

Duration	1–2 months
Core Topics	Lock-free programming, atomics, memory ordering, NUMA, cache locality, profiling, SIMD, thread pools
Projects	Profile and optimize a Phase 6 project with perf; benchmark before/after
Resources	Boost, Asio, fmt, spdlog

Phase 8 — OS Development (Build Your Own)

Duration	3–4 months
Core Topics	Bootloader, kernel, memory manager, scheduler, virtual memory, file system, shell, networking stack, drivers, optional GUI
Projects	A working toy OS with at least bootloader → kernel → scheduler → simple file system
Resources	OSDev Wiki; "Writing a Simple Operating System from Scratch"

Phase 9 — Backend at Scale

Duration	1–2 months
Core Topics	Distributed systems, consensus (Raft/Paxos), CAP theorem, Kubernetes, Docker, Kafka, Nginx, Envoy
Projects	Distributed key-value store, reverse proxy, message queue, mini database
Resources	Designing Data-Intensive Applications (recommended companion read)

Phase 10 — Security Specialization (Differentiator)

Duration	Ongoing, woven throughout rather than saved for last
Core Topics	Binary exploitation, secure coding in C++, memory corruption, fuzzing, reverse engineering, kernel exploitation, driver security, sandboxing, browser internals, EDR development
Projects	Mini antivirus, PE/ELF parser, memory scanner, debugger, disassembler, fuzzer, kernel module
Resources	Apply your existing red team background directly — build these as soon as the underlying CS concept (e.g. ELF format in Phase 3) is covered

Portfolio Projects (18–24 Month Target List)

- Custom shell
- HTTP server & multithreaded web server
- TCP chat server
- HTTP proxy
- Redis clone
- SQLite clone
- Linux process monitor
- Packet sniffer
- ELF parser / PE parser
- Debugger and disassembler
- Kernel module
- Custom memory allocator
- Toy operating system
- Distributed key-value store
- Bare-metal STM32 driver suite (GPIO, UART, SPI, I2C, no HAL)
- Hand-rolled preemptive scheduler for Cortex-M (mini RTOS)
- CAN bus sniffer / logger
- Firmware reverse-engineering / secure boot bypass lab (security crossover)

Technologies to Master

Languages	C++, C, Python, Go (later), x86-64 Assembly
Build Tools	CMake, Make, Ninja
Version Control	Git
Debugging	GDB, LLDB
Profiling	perf, Valgrind, AddressSanitizer (ASan), ThreadSanitizer (TSan)
Platforms	Linux, POSIX, systemd
Networking	BSD sockets, Boost.Asio

Databases	PostgreSQL, SQLite
Containers	Docker, Kubernetes (later)
Embedded Toolchain	arm-none-eabi-gcc, OpenOCD, JTAG/SWD, STM32CubeIDE (optional), FreeRTOS
Embedded Protocols	UART, SPI, I2C, CAN bus

Closing Note

Three legs, one discipline: Operating Systems gives you the theory of how a computer manages itself, Embedded Systems forces you to apply that theory with no safety net (no MMU, no scheduler handed to you, real memory limits), and Backend Engineering lets you apply the same core ideas at the opposite end of the resource spectrum. Let your security background be the thread that ties all three together — that combination, especially with a genuine embedded/firmware security angle, is uncommon and opens doors to specialized roles in systems, infrastructure, automotive/IoT security, and security engineering broadly.